Para construir um Micro SaaS real, escalável e de baixo custo, precisamos escolher ferramentas que o mercado sênior exige, mas sem criar uma complexidade desnecessária (*overengineering*). O segredo de um Engenheiro Sênior é entregar o máximo de valor com o menor custo operacional possível.
Aqui está a estrutura de pastas, tecnologias e ferramentas recomendadas para o seu projeto de tratamento de pele.

🛠️ Stack Tecnológica e Ferramentas

Frontend & Infraestrutura Web

- **Framework: Next.js 15+ (React)** com TypeScript. É o padrão de mercado atual. Permite Server Components (SEO excelente para seu produto público) e Client Components (para a área logada do paciente).
- **Estilização: Tailwind CSS** + **Shadcn/ui** (componentes prontos, limpos e fáceis de customizar para uma identidade visual de estética/saúde).
- **Hospedagem Frontend: Vercel** ou **Netlify** (plano gratuito atende perfeitamente o início).

Backend & Mensageria

- **API Gateway / BFF: Node.js (Fastify ou NestJS)** com TypeScript. Responsável por autenticação, upload de fotos e rotas rápidas do aplicativo.
- **Core Worker: .NET 9 (C#)** rodando como um *Background Service* (Worker Service). Responsável por gerar os cronogramas pesados, processar imagens e integrar com APIs de IA.
- **Mensageria Local: RabbitMQ** via Docker para desenvolvimento.
- **Mensageria Produção: AWS SQS** ou **CloudAMQP** (plano gratuito de RabbitMQ).

Banco de Dados & Storage

- **Banco Principal: PostgreSQL** (Hospedado na Neon.tech ou Supabase, ambos possuem camadas gratuitas excelentes).
- **Cache e Lock: Redis** (Hospedado na Upstash, focado em Serverless/Micro SaaS).
- **Storage de Fotos: AWS S3** ou **Cloudflare R2** (R2 possui taxa zero de transferência de dados, ideal para fotos de evolução de pele).

📂 Estrutura de Repositórios e Pastas

Para um Micro SaaS, a abordagem de **Monorepo** (tudo em um único repositório Git) facilitará muito sua vida no deploy e no desenvolvimento ágil. Usaremos o **Turborepo** ou **NPM Workspaces** para gerenciar.

```
meu-micro-saas/
├── .github/workflows/       # CI/CD (GitHub Actions para deploy automático)
├── apps/
```

```
│   ├── frontend/             # Aplicação Next.js (Interface do Usuário)
│   ├── api-node/             # Gateway / BFF em Node.js (TypeScript)
│   └── worker-dot-net/       # Processamento assíncrono em .NET 9 (C#)
├── packages/                 # Códigos compartilhados (opcional)
│   └── shared-types/         # Contratos de DTOs comuns entre Node e Frontend
├── docker-compose.yml        # Sobe PostgreSQL, Redis e RabbitMQ localmente
└── README.md                 # Arquitetura detalhada (seu portfólio sênior)
```

🏗️ Arquitetura Interna dos Serviços (Padrões Sênior)

1. API Node.js (TypeScript)

Evite bagunça. Use uma estrutura baseada em **Camadas (Controller -> Service -> Repository)** ou **Vertical Slices**(dividir por funcionalidades como `/payments`, `/users`, `/evaluations`).

- **ORM: Prisma** ou **Drizzle ORM** (Drizzle está em altíssima demanda pela performance extrema).
- **Validação: Zod** (para validar os dados que entram na API e garantir tipagem estática).

2. Worker .NET 9 (C#)

Aqui você demonstra senioridade aplicando **Clean Architecture** (Arquitetura Cebola) reduzida para o cenário de um Worker:

```
worker-dot-net/
├── src/
│   ├── Application/          # Casos de uso (Use Cases) e Handlers de Mensagens
│   ├── Domain/               # Entidades de negócio, Regras puras e Interfaces
│   ├── Infrastructure/       # Implementação de Banco de Dados, SQS/RabbitMQ, OpenAI API
│   └── Worker/               # O entrypoint que herda de BackgroundService (Program.cs)
```

- **Biblioteca de Mensageria: MassTransit**. Ela abstrai o RabbitMQ/SQS. Se você quiser trocar de fila no futuro, muda apenas uma linha de configuração.
- **ORM: Entity Framework Core** ou **Dapper** (se quiser focar em queries ultra rápidas de leitura).

🔄 Fluxo de Dados Prático (Onde a Mensageria Entra)

Para fixar o aprendizado em mensageria, veja o fluxo de quando um cliente envia uma foto para análise de celulite:

1. O **Frontend** gera uma URL assinada (*Presigned URL*) e faz o upload da foto direto para o **Cloudflare R2** (não sobrecarrega seu backend).
2. O **Frontend** envia os dados para a **API Node.js** dizendo: "*Foto enviada com sucesso no caminho X*".
3. A **API Node.js** publica uma mensagem no **RabbitMQ/SQS** chamada `PeleAnaliseSolicitadaEvent` contendo o `UserId` e o `ImageId`. A API responde `200 OK` na hora para o usuário.
4. O **Worker .NET** está escutando essa fila. Ele recebe o evento, baixa a imagem, chama a API da OpenAI Vision para categorizar o grau da celulite, salva o resultado no **PostgreSQL** e limpa o cache no **Redis**.
5. O **Worker** envia uma notificação (via *Websockets* ou *Push Notification*) avisando o Frontend que o relatório está pronto.